

Software Project Management Plan for the Undergraduate Transfer Management System (UTMS)

Prepared by:

Team 3

Organization: Izmir Institute of Technology (IZTECH) - Department of Computer Engineering

Course: SEDS 505

Date Created: November 19, 2025

1. Overview.....	3
1.1 Purpose, scope, and objectives.....	3
1.2 Assumptions and constraints.....	3
1.3 Project deliverables.....	4
1.4 Schedule and budget summary.....	4
2. References.....	4
3. Definitions.....	5
4. Project Context.....	6
4.1 Product acceptance plan.....	6
4.2 Project organization.....	6
4.2.1 External interfaces.....	6
4.2.2 Internal structure.....	7
4.2.3 Roles and responsibilities.....	7
5. Project Planning.....	8
5.1 Project work plans.....	8
5.1.1 Work activities.....	8
5.1.2 Schedule allocation.....	9
5.1.3 Resource allocation.....	9
5.1.4 Budget allocation.....	9
5.1.5 Communications plan.....	10
5.2 Project assessment and control.....	10
5.2.1 Schedule control plan.....	10
5.2.2 Budget control plan.....	10
6. Product Delivery.....	11

1. Overview

1.1 Purpose, scope, and objectives

The Undergraduate Transfer Management System (UTMS) is a web-based information system designed to automate the undergraduate transfer (*yatay geçiş*) process at IZTECH. The primary purpose of this project is to replace existing manual, paper-based procedures with a centralized, digital workflow that connects Students, the Student Affairs Office (ÖİDB), the Dean's Office, and the Transfer Commission (YGK).

The scope of the project includes the development of a secure web application that facilitates:

- Digital submission and tracking of transfer applications.
- Automated validation of academic eligibility and YKS exam scores.
- Controlled routing of applications between administrative bodies (ÖİDB, Dean's Office, Departments).
- Transparent evaluation, ranking, and decision publication.

The objective is to deliver a fully functional system that ensures accurate eligibility verification, enforces quota rules, and provides auditability for all transfer decisions.

1.2 Assumptions and constraints

Assumptions:

1. All system users (Students, Staff, Faculty) possess active institutional credentials for authentication.
2. A stable internet connection is available for all actors during the application and evaluation periods.
3. Evaluation rules (e.g., minimum GPA, conversion tables, quota formulas) are preconfigured and available before the start of the development cycle.
4. Faculty and department structural data exist and are accessible.

Constraints:

1. **Regulatory Compliance:** The system must strictly adhere to IZTECH data-handling regulations and Council of Higher Education (YÖK) rules.
2. **Security:** The application must be hosted on servers using HTTPS (SSL/TLS 1.3).
3. **Integration:** Integration is strictly limited to the internal University Information System (UBYS) API for student record data.
4. **Operational Window:** The application submission features must only be active during officially announced transfer periods.

1.3 Project deliverables

The following artifacts will be delivered to the client (IZTECH CENG Department / Course Instructors) upon project completion:

Artifact	Description
UTMS Web Application	The fully functional, deployed software system.
Source Code	Complete codebase hosted in the project repository (e.g., GitHub/GitLab).
SRS Document	Software Requirements Specification (Final Version).
SPMP Document	Software Project Management Plan (This document).
SDD Document	Software Design Description (Architecture and DB Design).
User Manuals	Separate guides for Students, ÖİDB Officers, and YGK Members.
Test Reports	Results of Unit, Integration, and System testing.

1.4 Schedule and budget summary

The project is scheduled for **16 weeks**.

- **Start Date:** October 1, 2025
- **End Date:** January 21, 2026
- **Estimated Effort:** 480 Man-Hours (Assumes 2 team members x 15 hours/week x 16 weeks).
- **Budget:** Zero monetary budget; costs are calculated in effort hours and usage of existing infrastructure.

2. References

1. **SRS:** *Undergraduate Transfer Management System (UTMS) - Software Requirements Specification*, Team 3, October 2025.
2. **IEEE Std 830-1998:** *IEEE Recommended Practice for Software Requirements Specifications*.
3. **IEEE Std 1058-1998:** *IEEE Standard for Software Project Management Plans*.
4. **IZTECH Regulations:** *IZTECH Undergraduate Transfer Regulation Documents* (ders_katalog.pdf, intibak.pdf, etc., as cited in SRS 1.4).

3. Definitions

The following acronyms and terms are used throughout this plan, consistent with the SRS.

Term	Definition
UTMS	Undergraduate Transfer Management System
ÖİDB	Öğrenci İşleri Daire Başkanlığı – Student Affairs Office
YGK	Yatay Geçiş Komisyonu – Transfer Commission
Dean’s Office	Faculty-level administrative office managing academic processes
UBYS	University Information System used for academic record exchange
Intibak	Course-equivalence (credit transfer) process
YKS	National University Entrance Exam
FR/NFR	Functional Requirement / Non-Functional Requirement

4. Project Context

4.1 Product acceptance plan

Product acceptance will be determined by the successful verification of all Functional Requirements (FR-01 through FR-20) listed in the SRS Section 4.4.

- **Verification Method:** System Testing and User Acceptance Testing (UAT).
- **Acceptance Criteria:**
 - All "Normal Courses" in Use Cases (UC-STU, UC-OIDB, UC-DEAN, UC-YGK, UC-ADM) function without critical errors.
 - Performance metrics (e.g., page load < 3s, support for 100 concurrent users) are met.
 - Security audits confirm RBAC and encryption standards (SEC-01 to SEC-09).

4.2 Project organization

4.2.1 External interfaces

The project interacts with the following external entities:

- **UBYS (University Information System):** The UTMS communicates via a secure API to retrieve student transcripts and push accepted transfer records.
- **IZTECH IT Department:** Provides hosting infrastructure and SSL certificates.
- **Council of Higher Education (YÖK):** Indirect interface via regulatory compliance (GPA conversion rules).

4.2.2 Internal structure

The project team (Team 3) operates as a completely flat, agile unit. Given the small team size (2 members) and the comprehensive nature of the project, both members act as **Co-Founders/Full Stack Engineers**.

There is no rigid separation between "Frontend" and "Backend." Both members participate in all phases of the SDLC, including database design, API implementation, UI development, testing, and deployment. Decisions are made by consensus, and tasks are assigned based on immediate availability rather than fixed roles.

4.2.3 Roles and responsibilities

The following table maps the project stakeholders to their roles. Note that the development team shares all technical responsibilities.

Project Role	SRS Actor / Team Member	Responsibility
Client / User Representative	ÖİDB Officer	Provides requirements for validation workflows; validates final transfer lists.
Client / User Representative	Dean's Office Staff	Provides requirements for routing and approval workflows.
Client / User Representative	YGK Member	Validates the evaluation logic and ranking algorithms.
End User	Student	Primary user for application submission and status tracking.
System Owner	System Administrator	Managing user roles and configuration (quotas/rules).
Full Stack Engineer	Egehan Vardar	Development (FE/BE/DB), DevOps, Project Management, Architecture.
Full Stack Engineer	Mehmet Fatih Gülmez	Development (FE/BE/DB), Quality Assurance, Documentation, UI Design.

5. Project Planning

5.1 Project work plans

5.1.1 Work activities

The project work is decomposed into 8 Work Packages (WP), further broken down into specific tasks. This Work Breakdown Structure (WBS) ensures the 2-person team addresses all functional modules from the SRS.

WP-1: Project Management & Analysis

- 1.1 SRS Refinement & Sign-off
- 1.2 SPMP Creation (Schedule & Resource Planning)
- 1.3 Setup Collaboration Tools (GitHub, Discord)

WP-2: System Design & Architecture

- 2.1 Database Design (ER Diagram normalization & SQL Schema generation)
- 2.2 API Specification (OpenAPI/Swagger definitions)
- 2.3 UI/UX Design (Wireframes for critical paths)

WP-3: Implementation Phase 1 - Core & Student Module

- 3.1 Project Scaffolding (Repo setup, CI pipeline init)
- 3.2 Authentication Service (Login, Logout, JWT Implementation)
- 3.3 Student Application Form (Frontend & Backend)
- 3.4 Document Upload Service (File storage logic)

WP-4: Implementation Phase 2 - Administrative Workflows

- 4.1 ÖİDB Dashboard (List views, Validation logic)
- 4.2 Dean's Office Module (Routing logic)
- 4.3 Status Notification System (Email triggers)

WP-5: Implementation Phase 3 - Evaluation Engine

- 5.1 YGK Evaluation Interface (Review screen)
- 5.2 Auto-Scoring Algorithm & Ranking Logic
- 5.3 Final Decision Approval Workflow

WP-6: Implementation Phase 4 - System Configuration & Public View

- 6.1 Admin Console (User Management)
- 6.2 Rule Engine Configurator (Quotas/GPA thresholds)
- 6.3 Announcement Module (Public results page)

WP-7: Quality Assurance & Testing

- 7.1 Unit Testing (Critical logic coverage)
- 7.2 Integration Testing (Mocking external APIs)
- 7.3 User Acceptance Testing (Scenario runs)

WP-8: Deployment & Handover

- 8.1 Dockerization & Server Deployment
- 8.2 Documentation (User Manuals & Tech Docs)

- 8.3 Final Presentation Prep

5.1.2 Schedule allocation

The project follows a 16-week timeline from Oct 1 to Jan 21. Tasks are serialized to accommodate the 2-person team capacity.

Task ID	Work Package	Start Date	End Date	Duration	Notes
WP-1	Planning	Oct 01	Oct 14	2 Wks	Initial Setup
2.1, 2.2	DB & API Design	Oct 15	Oct 24	1.5 Wks	Joint Effort
2.3	UI Design	Oct 25	Oct 31	1 Wk	
WP-3	Dev: Core/Student	Nov 01	Nov 21	3 Wks	Milestone 1
4.1, 4.2	Dev: Admin Flows	Nov 22	Dec 12	3 Wks	
WP-5	Dev: Evaluation	Dec 13	Dec 31	2.5 Wks	Milestone 2
WP-6	Dev: Config/Public	Jan 01	Jan 07	1 Wk	
WP-7	Testing	Jan 08	Jan 15	1 Wk	Code Freeze
WP-8	Deploy & Docs	Jan 16	Jan 21	6 Days	Final Delivery

5.1.3 Resource allocation

The team employs a flexible resource-allocation strategy in which both members contribute across all technical layers (Frontend, Backend, Database, DevOps).

Personnel:

- **Egehan Vardar (Full Stack Engineer):**
 - *Responsibilities:* Works on both Frontend (React) and Backend (Java) tasks. Leads architectural decisions, server management (DevOps), and coordinates project schedule (PM duties).
- **Mehmet Fatih Gülmez (Full Stack Engineer):**
 - *Responsibilities:* Works on both Frontend (React) and Backend (Java) tasks. Leads QA efforts (Testing), documentation creation, and UI design.

Hardware & Software:

- **Development Hardware:** 2x Personal Workstation Laptops.
- **Server Hardware:** 1x Dedicated Server for Staging, Production, and CI/CD hosting.
- **Software Stack:** Java (Backend), React (Frontend), PostgreSQL (DB), Docker (Containerization).

5.1.4 Budget allocation

Budget is defined in **Man-Hours**.

- **Total Duration:** 16 Weeks.
- **Weekly Workload:** ~15 hours per person.
- **Total Capacity:** 2 people * 15 hrs * 16 wks = **480 Hours**.

Breakdown by Work Package:

Work Package	Estimated Hours	Assigned
WP-1: Planning	40 hrs	Joint
WP-2: Design	60 hrs	Joint
WP-3: Core Dev	100 hrs	Joint / Split by Task
WP-4: Admin Dev	80 hrs	Joint / Split by Task
WP-5: Eval Dev	80 hrs	Joint / Split by Task
WP-6: Config Dev	40 hrs	Joint / Split by Task
WP-7: Testing	50 hrs	Joint
WP-8: Deployment	30 hrs	Joint
TOTAL	480 Hours	

5.1.5 Communications plan

- **Internal Team:** Weekly sync meetings (Monday mornings) and daily stand-ups via Discord.
- **Instructor/Supervisor:** Bi-weekly progress reports and milestone demonstrations.
- **Documentation:** All decisions are recorded in the project repository Wiki.

5.2 Project assessment and control

5.2.1 Schedule control plan

Progress will be tracked against the WBS and Schedule Allocation (Section 5.1.2).

- **Mechanism:** Weekly reviews of "To Do", "In Progress", and "Done" tasks on the Kanban board.
- **Corrective Action:** If a milestone is delayed by >3 days, scope for non-critical features (e.g., advanced dashboard analytics) will be reduced to ensure the Critical Path (Application -> Evaluation -> Result) is delivered on time.

5.2.2 Budget control plan

Since the budget is effort-based, team members will log hours per task.

- **Quality Control Integration:** To ensure Quality Requirements (SRS Section 4) are met without blowing the effort budget:
 - **Code Reviews:** Mandatory for every Pull Request to ensure maintainability (MAI-06).
 - **Automated Testing:** CI pipeline to run tests on commit to catch regressions early, preserving debugging time.
 - **Performance Checks:** JMeter load tests run at Milestone 3 to ensure PRF-01 (3s load time) is met before final delivery.

6. Product Delivery

The final product will be delivered via the Course Submission Portal and a live demonstration.

1. **Packaging:** The application will be containerized (Docker) for consistent deployment (Requirement SCP-03).
2. **Documentation:** A zip archive containing the Source Code, SRS, SPMP, SDD, and User Manuals (PDF format).
3. **Deployment:** A live instance will be accessible on the simulated environment for the final presentation.